

Hybrid-INoT: a hybrid agent architecture with internal role segregation for engineering tasks under long shared context

ANONYMOUS AUTHOR(S)

External multi-agent orchestration repeatedly transmits shared context to separate large-language-model roles, increasing token usage. We propose **Hybrid-INoT**, a hybrid architecture in which the *planner-worker-critic-self-check* cycle is executed inside one model call, while tool actions, verification, and context compression remain external. We define its operational model, routing function, and analytical conditions for token and economic efficiency. A reproducible evaluation uses HumanEval and a controlled task set. In a real Gemini 3.1 Pro Preview API pilot ($n = 5$ tasks per context length), Hybrid-INoT reduced mean token consumption relative to Classical-MAS by 55.6–88.9% as target context grew from 256 to 16,384 tokens; an independent Gemini 3.1 Flash-Lite run ($n = 20$, seed 42) yielded 49.6–90.3% savings. In the latter run, the paired U_{tok} gain was significant at all four context lengths after Holm–Bonferroni correction ($p = 1.91 \cdot 10^{-6}$). However, pass@1 was 1.0 for both architectures, so the stated quality non-inferiority margin was not statistically established. The small-model profitability condition held in the HumanEval pilot but not in the approximate SWE-bench Lite protocol. An analytical extrapolation of observed Pro API costs gives \$429.96 savings per 10,000 tasks under an equal mixture of four context lengths (95% bootstrap CI \$267.06–\$617.17); this is not a full-TCO estimate. The evidence therefore supports token efficiency in the studied range but does not establish quality preservation, transfer to real repositories, or lower total cost of ownership.

CCS Concepts: • **Software and its engineering** → **Automatic programming**; • **Computing methodologies** → *Multi-agent systems*.

Additional Key Words and Phrases: Large language models, agent systems, Introspection of Thought, token efficiency, inference economics, HumanEval, reproducible experiment

ACM Reference Format:

Anonymous Author(s). 2027. Hybrid-INoT: a hybrid agent architecture with internal role segregation for engineering tasks under long shared context. In *ACM International Conference on the Foundations of Software Engineering (FSE 2027)*, 12–16 July 2027, Shenzhen, China. ACM, New York, NY, USA, 16 pages.

1 Introduction

A modern software-engineering agent must successively understand the task specification, retrieve the relevant context from a repository, and construct a change that can then be checked by tests or static analysis. In practice, these functions are often distributed across several roles: one role builds a plan, another proposes a solution, a third critiques the intermediate result, and a fourth runs external verification. This decomposition is conceptually convenient, but it becomes expensive when all roles are forced to reread the same long context.

This paper focuses precisely on that setting: tasks in which the code, documentation, compiler messages, and execution logs are shared across all roles. We are interested not only in the final solution quality but also in the cost of the chosen orchestration scheme in tokens, response time, and monetary spending. The article therefore compares classical external multi-agent organization with a hybrid scheme in which part of the role discussion is moved inside a single model call.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

FSE 2027, Shenzhen, China

© 2027 Copyright held by the owner/author(s). Publication rights licensed to ACM.

1.1 The problem of repeated context transmission and its practical significance

In agent systems built on large language models, a typical configuration is external multi-agent orchestration (Classical-MAS): separate processes or calls implement the planner, worker, critic, and tester roles [6, 7, 19]. In software-engineering tasks, where several roles read the same long fragments of code, logs, and documentation, this scheme has a structural defect: *each role receives the shared context C_0 anew*, and additional coordination messages circulate between roles. In token-budget terms, this yields the term

$$D_{\text{ctx}} + H_{\text{coord}} \in \Theta(|\mathcal{A}| \cdot |C_0|) + \Theta(|\mathcal{A}|^2 \cdot \bar{h}),$$

where $|\mathcal{A}|$ is the number of roles and $\bar{h}$ is the average coordination-message length. In our API pilot, this structural effect was observed directly. At a target context of 16,384 tokens, Classical-MAS used 59,631 tokens per task on average, of which 57,485 (96.4%) were input tokens; Hybrid-INoT used 6,608 tokens. Per-case cost was \$0.14072 and \$0.02949, respectively, at the recorded Gemini 3.1 Pro Preview prices [16]. These values concern the first five HumanEval tasks and must not be generalized to a different task distribution without measurement.

The problem is therefore quantitatively observable: in the studied long-context regime, repeated input transmission produced an approximately ninefold difference in total tokens between B2 and B3. The remainder of the paper separates the analytical model from empirical evidence and restricts claims to completed pilots.

1.2 Existing approaches and their limitations

The open literature contains three related lines of work: (i) single-model in-context self-critique, such as Self-Refine [13] and Reflexion [22]; (ii) alternation between reasoning and action inside one agent, such as ReAct [30] and SWE-agent [29]; and (iii) external role decomposition into separate calls, such as MetaGPT [7] and ChatDev [19]. None of these works defines an *operationally distinct* intermediate configuration in which (a) the shared context is preserved inside one call without duplication, (b) role segregation remains sufficient for critical examination, and (c) only operations with externally verifiable signals, such as tests, static analysis, and tool calls, remain external. The literature also lacks a formal condition under which such a hybrid dominates a purely external Classical-MAS scheme in expected token consumption.

1.3 Main results and contribution

The paper delivers the following results:

- C1. An operational definition of Hybrid-INoT with an explicit state signature, the RouteMode routing function, and a termination condition for the internal role cycle (§4).
- C2. Proposition 2, which states the token advantage of Hybrid-INoT over Classical-MAS in expectation when the shared context is sufficiently long, and proves it by an explicit token accounting under stated assumptions (§5).
- C3. Inequality (2), which gives the profitability condition for a small self-check model (§5).
- C4. A system of metrics for token efficiency, monetary cost, and code maintainability with a justified aggregation form (§6).
- C5. A reproducible real-API evaluation covering context-length scaling, ablations, the small-model check, and an E6 cross-model sensitivity analysis (§7).
- C6. Evidence-aligned hypothesis verdicts: support for the token component of H1 in the Flash-Lite run, partial support for H2 only on HumanEval, and no support for H3 in the direct architectural comparison (§7).

The harness, configurations, raw results, and figure-generation script are linked to an immutable research commit (§7).

2 Related work and distinction of the proposed approach

2.1 From chain-of-thought to internal role debate

Wei et al. [26] show that explicit verbalization of intermediate reasoning steps can improve solution quality on tasks that require several connected logical transitions. In this context, intermediate steps are textual fragments by which the model decomposes a complex inference into a sequence of local subproblems, while quality refers to the final success of solving the target task. The Self-Refine approach [13] formalizes a “generator–feedback–revision” cycle inside a single model; Reflexion [22] supplements such a scheme with reflective memory, whereas ReAct [30] couples internal reasoning with external actions and tools. Sun and Zeng [24] describe *Introspection of Thought* (INoT) as a structured internal debate among several roles. At the same time, Turpin et al. [25] show that intermediate chain-of-thought text should not be interpreted as a faithful description of the hidden computational process. Accordingly, in the present paper introspection is treated primarily as a computational mechanism for candidate selection, critique, and selective rerun rather than as a literal window into the latent state of the model.

2.2 Multi-agent systems for software engineering

Weiss [27] and Wooldridge [28] provide the theoretical framework for MAS. MetaGPT [7] and ChatDev [19] transfer this framework into the context of large language models: roles are implemented as separate calls, each receiving its own copy of the context. Survey work identifies two key limitations of Classical-MAS: error propagation along the chain and the growth of coordination cost [6, 20]. SWE-bench and SWE-agent [11, 29] further show that practical software-engineering automation requires long context and iterative editing, which makes the retransmission problem D_{ctx} even more acute.

2.3 Context compression and external verification

LongLLMLingua and LLMingua-2 [10, 17] demonstrate that prompt compression can preserve or even improve quality under long context while lowering cost. Jia et al. [9] apply this logic directly to code. For code self-check, linguistic reflection alone is insufficient: external observable signals are required, namely tests, static analysis, and compiler feedback [3, 21, 23].

2.4 Comparison of Hybrid-INoT with prior approaches

To systematize the differences among the architectures under discussion, we distinguish five features: context organization, the structure of role interaction, the verification mechanism, the way the execution mode is chosen, and the stopping condition.

Table 1 shows that individual components of the proposed scheme appear in earlier work, but it is precisely their combination inside a single internal role cycle, supplemented by external tool-based verification and explicit routing, that constitutes the Hybrid-INoT architecture.

3 Formal problem statement

3.1 Task

Let $\mathcal{X}$ denote the set of textual task specifications, $\mathcal{C}$ the set of finite sequences of contextual artifacts (code files, documentation fragments, and execution logs), and $\mathcal{Y}$ the set of candidate solutions representable as textual artifacts and amenable to external verification. Let $(\Omega, \mathcal{F}, \mathbb{P})$ be

Table 1. Comparison of Hybrid-INoT with adjacent architectural classes

Feature	Self-Refine / Reflexion	MetaGPT / ChatDev	Hybrid-INoT
Context organization	One shared context without explicit role separation.	A separate copy of the shared context is created for each role.	One shared context is preserved inside a model call and shared among the internal roles.
Role interaction	Self-critique is embedded in a single-model loop.	Roles are separated into distinct calls and exchange messages.	Planning, generation, and critique are executed as internal roles within one cycle.
Solution verification	Predominantly linguistic self-evaluation.	Predominantly role-based linguistic checking; external verification depends on the implementation.	The final candidate must pass external tool-based verification; linguistic critique is used as an intermediate filter.
Mode selection	Fixed by the method architecture.	Fixed by the method architecture.	Set by an explicit routing function that switches between internal and external modes.
Stopping condition	A fixed number of iterations or an episodic stopping rule.	Completion of the role pipeline or agreement among roles.	Stop after successful external verification or after the iteration limit is reached.

the probability space induced by stochastic model generation, seed choice, and nondeterminism in external tool calls.

DEFINITION 1 (AGENTIC ENGINEERING TASK). *An agentic engineering task is the tuple $(x, C_0, \mathcal{Z}, \mathcal{V})$, where $x \in \mathcal{X}$ is a textual task description; $C_0 \in \mathcal{C}$ is the initial task context, including code, documentation, and execution logs; $\mathcal{Z}$ is a finite set of permitted external tools; and $\mathcal{V} : \mathcal{Y} \rightarrow \{0, 1\}$ is an externally verifiable success function, for example, the execution of a test suite.*

The agent strategy induces a random output $Y : \Omega \rightarrow \mathcal{Y}$. At the same time, random variables $T : \Omega \rightarrow \mathbb{R}_{\geq 0}$, $C_{\text{inf}} : \Omega \rightarrow \mathbb{R}_{\geq 0}$, and $\ell : \Omega \rightarrow \mathbb{R}_{\geq 0}$ are defined, representing, respectively, the total token consumption, the monetary inference cost, and the latency of solving one task. The goal of the agent is to maximize $\mathbb{E}[\mathcal{V}(Y)]$ under the budget constraints $\mathbb{E}[T] \leq B$, $\mathbb{E}[C_{\text{inf}}] \leq B_{\$}$, and $\mathbb{E}[\ell] \leq B_{\ell}$.

Main notation.

C_0 – the initial shared task context; $|C_0|$ is its token length.

$\hat{C}$ – the compressed version of the initial context used in the internal cycle.

$\mathcal{A}$ – the set of external roles in the Classical-MAS scheme; $|\mathcal{A}|$ is the number of such roles.

Y – the random output of the agent with values in $\mathcal{Y}$; $y \in \mathcal{Y}$ is an individual candidate solution.

- K — the number of candidates generated by the role r_{work} in one iteration of the internal cycle.
 T — the random variable of total token consumption for one task; $\mathbb{E}[T]$ is taken with respect to the distribution defined on $(\Omega, \mathcal{F}, \mathbb{P})$.
 U_{tok} — the token-efficiency measure defined as the ratio of final solution quality to total token consumption.
 τ^* — the threshold initial-context length beyond which Hybrid-INoT has an expected token advantage.
 δ_{H1} — the minimal practically relevant gain in U_{tok} used in Hypothesis H1.
 C_{check}^S — the cost of the preliminary check performed by the small model.
 C_{rerun}^L — the cost of rerunning the large model.
 ρ_0, ρ_1 — the probabilities of an expensive rerun without and with the preliminary check, respectively.

3.2 Research hypotheses

- H1** (advantage under long context). There exists a threshold initial-context length τ^* such that for $|C_0| > \tau^*$, Hybrid-INoT yields at least a $\delta_{H1} = 15\%$ increase in U_{tok} relative to Classical-MAS, while pass@1 degrades by no more than 2 percentage points.
Verification: E3 (§7.2) with paired Wilcoxon tests, 95% bootstrap confidence intervals, and Holm–Bonferroni correction.
H2 (economic justification of the small self-check model). There exists a regime in which a small-model preliminary check reduces expensive reruns enough for $C_{\text{check}}^S < (\rho_0 - \rho_1)C_{\text{rerun}}^L$.
Verification: E4 (§7.4) on 20 HumanEval tasks and 20 approximate SWE-bench Lite cases.
H3 (boundary of applicability). On tasks with genuinely independent subtasks and parallel tools, the response-time advantage of Hybrid-INoT vanishes or becomes negative.
Verification: an architectural comparison on five tasks and a separate controlled serial/parallel tool microbenchmark in E2c (§7.3).

When testing H1, the base model, tools, external verification, task set, seed, temperature, and prompts are held fixed; only orchestration changes. The threshold $\delta_{H1} = 15\%$ is treated as the minimum practically meaningful effect. Observed savings do not replace evaluation of the complete H1 criterion.

4 Method: Hybrid-INoT

4.1 States and role signatures

Define the system state as $\sigma = (x, \hat{C}, \mathcal{H}, i, b)$, where $\hat{C}$ is the compressed context, $\mathcal{H}$ is the history of iterations, $i \in \{0, \dots, I\}$ is the iteration counter, and $b \in \mathbb{N}$ is the remaining token budget.

Let Π be the space of plans, and let $K \in \mathbb{N}$ denote the number of candidates generated by the role r_{work} in one iteration. Then each role is defined by a signature $r : \mathcal{S}_r^{\text{in}} \rightarrow \mathcal{S}_r^{\text{out}}$:

$$\begin{aligned}
 r_{\text{plan}} : (x, \hat{C}, b) &\mapsto \pi \in \Pi, \\
 r_{\text{work}} : (\pi, \hat{C}, K) &\mapsto \mathbf{y} = (y_1, \dots, y_K) \in \mathcal{Y}^K, \\
 r_{\text{crit}} : (\mathbf{y}, \hat{C}) &\mapsto (s_1, \dots, s_K) \in [0, 1]^K, \\
 r_{\text{self}} : (y \in \mathcal{Y}) &\mapsto (v, M) \in \{0, 1\} \times [0, 1].
 \end{aligned}$$

The roles $r_{\text{plan}}, r_{\text{work}}, r_{\text{crit}}$ are implemented as role prefixes inside *one* large-model call over the shared $\hat{C}$; the role r_{self} is an external process running tests and static analysis.

4.2 Routing function

DEFINITION 2 (ROUTEMODE). Let $\varphi_{\text{tool}}(x) \in \{0, 1\}$ indicate whether an external tool is mandatory, let $\varphi_{\text{par}}(x) \in \{0, 1\}$ indicate dominant subtask parallelism, and let $|\hat{C}|$ be the length of the compressed context. Then

$$\text{RouteMode}(x, \hat{C}) = \begin{cases} \text{external}, & \varphi_{\text{tool}}(x) = 1 \vee \varphi_{\text{par}}(x) = 1 \vee |\hat{C}| < \tau_{\text{route}}^*, \\ \text{internal}, & \text{otherwise.} \end{cases}$$

The parameters φ_{tool} and φ_{par} are determined by a separate classifier over x (a lightweight instruction-based classifier in the base implementation). The threshold $\tau_{\text{route}}^* = 1024$ tokens remains an engineering setting and must not be conflated with the left-censored research result $\tau^* \leq 256$ from E3.

4.3 Internal cycle and termination condition

Algorithm 1 Hybrid-INoT: internal introspective cycle

Require: task x , context C_0 , budget B , maximum number of iterations I , number of candidates K

Ensure: solution y^* , flag v^* , score M^*

```

1:  $C \leftarrow \text{PolicySanitize}(x, C_0)$ 
2:  $\hat{C} \leftarrow \text{CompressIfNeeded}(C)$  ▷ Alg. 2
3:  $m \leftarrow \text{RouteMode}(x, \hat{C})$  ▷ Def. 2
4: if  $m = \text{external}$  then return  $\text{ExternalAgentWorkflow}(x, \hat{C})$ 
5: end if
6:  $s^{\text{best}} \leftarrow -\infty$ ;  $y^{\text{best}} \leftarrow \perp$ 
7: for  $i = 1$  to  $I$  do
8:    $\pi_i \leftarrow r_{\text{plan}}(x, \hat{C}, B - \text{spent})$ 
9:    $\{y_{i,k}\} \leftarrow r_{\text{work}}(\pi_i, \hat{C}, K)$ 
10:   $(s_{i,1}, \dots, s_{i,K}) \leftarrow r_{\text{crit}}(\{y_{i,k}\}, \hat{C})$ 
11:   $k^* \leftarrow \arg \max_k s_{i,k}$ ;  $y_i \leftarrow y_{i,k^*}$ 
12:  if  $s_{i,k^*} > s^{\text{best}}$  then  $s^{\text{best}} \leftarrow s_{i,k^*}$ ;  $y^{\text{best}} \leftarrow y_i$ 
13:  end if
14:   $(v_i, M_i) \leftarrow r_{\text{self}}(y_i)$  ▷ external tests and static analysis
15:  if  $v_i = 1$  then return  $(y_i, 1, M_i)$ 
16:  end if
17:   $\hat{C} \leftarrow \text{UpdateContextWithFailures}(\hat{C}, v_i)$ 
18:  if  $\text{stagnation}(s_{i,k^*} - s_{i-1,k^*}) < \varepsilon$  then break ▷ see Prop. 1
19:  end if
20: end for
21: return  $(y^{\text{best}}, 0, M(y^{\text{best}}))$ 

```

PROPOSITION 1 (TERMINATION OF THE INTERNAL CYCLE). Assume that each operation within one iteration of Algorithm 1 terminates in finite time and that the number of iterations is bounded above by the parameter I . Then the algorithm terminates no later than after I iterations: it either returns a solution immediately after a successful external check $v_i = 1$, or stops earlier under the rule $\text{stagnation}(s_{i,k^*} - s_{i-1,k^*}) < \varepsilon$, or, after exhausting the iteration limit, returns the best candidate found.

PROOF. The outer loop over the index i contains at most I passes. Each iteration consists of a finite sequence of steps: plan construction, generation of K candidates, critical scoring, external verification, and context update. By assumption, all of these operations terminate in finite time;

therefore, each iteration also terminates. Consequently, the algorithm either stops early by the instructions **return** or **break**, or reaches the final **return** after the I -th iteration. Hence the running time of the internal cycle is bounded above by a finite number of iterations, which proves the statement. $\square$

Note. Proposition 1 guarantees only algorithm termination; it does not imply monotonic growth of the scores s_i , nor does it guarantee the correctness of the critic itself. The scenario in which the critic systematically endorses incorrect solutions is examined separately in E2b (§7.3) as one of the threats to validity.

4.4 Context compression

Algorithm 2 Compression of long context

Require: C_0 , length threshold τ_{len}

Ensure: $\hat{C}$

- 1: **if** $|C_0| \leq \tau_{\text{len}}$ **then return** C_0
 - 2: **end if**
 - 3: split C_0 into semantic blocks $\{b_j\}$
 - 4: estimate $\text{rel}(b_j)$ using a classifier of the LLMlingua-2 type [17]
 - 5: remove duplicates, low-information logs, and secondary documentation
 - 6: retain API signatures, failing traces, editable files, and task constraints
 - 7: **return** $\hat{C}$ with $|\hat{C}| \leq \tau_{\text{len}}$
-

4.5 Selective rerun

Algorithm 3 Selective rerun based on verified signals

Require: candidates $\{y_1, \dots, y_K\}$, scores $\{s_k\}$, rerun budget R

Ensure: y^*

- 1: sort $\{y_k\}$ in descending order of s_k ; keep the top $M \leq K$
 - 2: **for each** y_m **do**
 - 3: $q_m \leftarrow \text{ExternalVerify}(y_m)$
 - 4: **if** $q_m = 1$ **then return** y_m
 - 5: **end if**
 - 6: **end for**
 - 7: **if** $R > 0$ **then** construct the prompt only from verified failure signals; **return** $\text{SelectiveRerun}(R - 1)$
 - 8: **end if**
 - 9: **return** $\arg \max_k s_k$
-

5 Analytical model of token and monetary costs

5.1 Structure of the analytical relations

The analysis in this section includes two types of relations. The first comprises direct balance equalities and inequalities that define the structure of token and monetary costs. The second comprises statements comparing architectures under fixed assumptions about context length, coordination cost, and internal critique. This separation makes it possible to distinguish general accounting relations from results that require additional assumptions.

5.2 Token budgets of Classical-MAS and Hybrid-INoT

ASSUMPTION 1. Among the roles in $\mathcal{A}$ under the Classical-MAS scheme, at least $\alpha|\mathcal{A}|$ roles, with $\alpha \in (0, 1]$, read the shared context C_0 in full.

ASSUMPTION 2. The length of coordination messages is bounded by $\bar{h}$, the number of iterations in both schemes is I , the compression cost satisfies $T_{\text{cmp}} \leq \gamma|C_0|$ with $\gamma < \alpha|\mathcal{A}|$, and the additional cost of internal critique satisfies $T_{\text{core}}^{\text{add}} \leq \beta|\hat{C}|$ with $\beta < \alpha|\mathcal{A}| - \gamma$.

PROPOSITION 2 (TOKEN ADVANTAGE UNDER LONG CONTEXT). Under Assumptions 1–2 and $|\mathcal{A}| \geq 3$, there exists $\tau^* > 0$ such that for $|C_0| > \tau^*$,

$$\mathbb{E}[T_{\text{MAS}}] - \mathbb{E}[T_{\text{INoT}}] \geq (\alpha|\mathcal{A}| - \gamma - \beta) \cdot |C_0| + \Theta(|\mathcal{A}|^2 \bar{h}) - T_{\text{check}} - \mathbb{E}[T_{\text{rerun}}] > 0.$$

DERIVATION FROM THE ASSUMPTIONS. Let T_{gen} denote the aggregate token contribution of candidate generation and other terms that do not depend linearly on the number of external roles. Then the first inequality follows from a direct decomposition of the Classical-MAS token budget into three parts: repeated transmission of the shared context, coordination exchange among roles, and candidate generation. By Assumption 1, in each of the I iterations, at least $\alpha|\mathcal{A}|$ external roles reread the full context C_0 , so

$$T_{\text{ctx}}^{\text{MAS}} \geq I \cdot \alpha|\mathcal{A}| \cdot |C_0|.$$

By Assumption 2, coordination messages contribute an additional term of order $I \cdot |\mathcal{A}|^2 \bar{h}$. Therefore,

$$\mathbb{E}[T_{\text{MAS}}] \geq I \cdot \alpha|\mathcal{A}| \cdot |C_0| + I \cdot |\mathcal{A}|^2 \bar{h} + T_{\text{gen}}.$$

For Hybrid-INoT, the shared context is compressed once, so $T_{\text{cmp}} \leq \gamma|C_0|$; the additional cost of internal critique is bounded above by $T_{\text{core}}^{\text{add}} \leq \beta|\hat{C}| \leq \beta|C_0|$. The external check and possible reruns contribute the terms T_{check} and $\mathbb{E}[T_{\text{rerun}}]$, while the remaining generation contribution is again absorbed into T_{gen} . Hence

$$\mathbb{E}[T_{\text{INoT}}] \leq I \cdot |\hat{C}| + T_{\text{cmp}} + T_{\text{core}}^{\text{add}} + T_{\text{check}} + \mathbb{E}[T_{\text{rerun}}] + T_{\text{gen}}, \quad |\hat{C}| \leq |C_0|.$$

Substituting these bounds and grouping the terms that depend linearly on $|C_0|$ yields a lower bound on the difference between the token budgets. For sufficiently large $|C_0|$, the linear term with a positive coefficient compensates the constant additions, which establishes the existence of the threshold τ^* . $\square$

Scope of applicability. Proposition 2 gives a lower bound on the difference between the expected token budgets under Assumptions 1–2. When $|\mathcal{A}| = 2$ (for example, a plan+exec configuration without a separate critic), the sign of this difference may change. Likewise, if $|C_0| < \tau^*$, an advantage of Hybrid-INoT is not guaranteed. The value of τ^* depends on $\alpha, \beta, \gamma, |\mathcal{A}|$, and I , and must therefore be refined empirically.

5.3 Worst-case upper bound on token usage

PROPOSITION 3 (WORST-CASE UPPER BOUND ON HYBRID-INoT TOKEN USAGE). In the worst case, when all I iterations end with $v_i = 0$,

$$T_{\text{INoT}}^{\text{worst}} \leq T_{\text{cmp}} + I \cdot K \cdot (p^L + r^L) + I \cdot T_{\text{check}} + I \cdot T_{\text{rerun}}^{\text{unit}}.$$

The resulting estimate defines an upper bound on additional token costs and is used as a budget restriction under B : as $T \rightarrow B$, Algorithm 1 returns the best candidate available.

5.4 Profitability condition for the small self-check model

Inference cost:

$$C_{\text{inf}} = c_L^{\text{in}} \sum_i p_i^L + c_L^{\text{out}} \sum_i r_i^L + c_S^{\text{in}} \sum_i p_i^S + c_S^{\text{out}} \sum_i r_i^S. \quad (1)$$

A small self-check model is economically beneficial if and only if

$$C_{\text{check}}^S < (\rho_0 - \rho_1) \cdot C_{\text{rerun}}^L. \quad (2)$$

The left-hand side of the inequality is the cost of the preliminary check, whereas the right-hand side is the expected savings from reducing the probability of an expensive rerun of the large model. Inequality (2) therefore defines the decision criterion between a cheap preliminary check and a rerun of the large model. Pilot measurements of ρ_0 , ρ_1 , C_{check}^S , and C_{rerun}^L are reported in E4.

6 Metric system

6.1 Main metrics and the rationale for the aggregation form

$$U_{\text{tok}} = \frac{Q}{T}, \quad Q_{\text{\$}} = \frac{Q}{C_{\text{inf}}}, \quad \text{VCR} = \frac{1}{N} \sum_i \mathbb{I}[\text{tests}_i = 1 \wedge \text{static}_i = 1 \wedge \text{sec}_i = 1]. \quad (3)$$

For heterogeneous quality, the paper uses a weighted sum over task classes, $Q = \sum_b w_b Q_b$, with weights specified in Table 2.

6.2 Aggregated success metric and its form

The aggregated success metric is defined as

$$\text{MAS}_i = \min(\text{Success}_i, \text{Verified}_i) \cdot M_i^\lambda, \quad \lambda \in [0, 1], \quad \lambda = 0.5. \quad (4)$$

The rationale for this construction is as follows. First, Success and Verified are necessary conditions for counting a solution as successful: a patch that fails external verification should not receive a high final score. Second, the multiplier M_i^λ models the effect of maintainability on the final assessment; the parameter λ controls the strength of this effect. The baseline uses $\lambda = 0.5$; E2d produced seven ranking inversions under weight and λ changes, so the composite metric is treated as secondary.

6.3 Maintainability weights and their rationale

$$M_i = w_{\text{cc}}(1 - \widehat{\text{CC}}_i) + w_{\text{dup}}(1 - \widehat{\text{Dup}}_i) + w_{\text{warn}}(1 - \widehat{\text{Warn}}_i) + w_{\text{doc}}\widehat{\text{Doc}}_i. \quad (5)$$

Table 2. Maintainability weights and their relation to ISO/IEC 25010:2023 subcategories [8]

Parameter	Value	Rationale
w_{cc}	0.35	ISO 25010 Modifiability; cyclomatic complexity [14] is the most direct proxy for maintenance cost.
w_{dup}	0.25	ISO 25010 Reusability; duplication is a key signal of technical debt.
w_{warn}	0.20	ISO 25010 Analysability; warnings are a surface-level yet reliable signal.
w_{doc}	0.20	ISO 25010 Modifiability; documentation coverage is used as a proxy for comprehensibility.

Sensitivity analysis. E2d tested the robustness of the configuration ranking under $(w_{\text{cc}}, w_{\text{dup}}, w_{\text{warn}}, w_{\text{doc}}) \pm 20\%$ and $\lambda \in \{0, 0.25, 0.5, 0.75, 1\}$ in (4); seven reversals were observed.

6.4 Reliability metrics

$$\text{SCR} = \frac{\#\{\text{incorrect candidates detected by the self-check procedure}\}}{\#\{\text{all incorrect candidates}\}}, \quad (6)$$

$$\text{RFR} = \frac{\#\{\text{patches without rollback}\}}{\#\{\text{applied patches}\}}. \quad (7)$$

7 Empirical validation

7.1 Reproducibility, source data, and statistical protocol

The research harness and the immutable experimental snapshot are supplied in the anonymous replication package [1]; every result below is tied to that frozen snapshot. The built-in configuration check completed successfully, and the test suite reported 15 passing tests. Raw JSON inputs are stored under `results/slide_e2e3_real`, `results/e6`, and `reproducibility/results/20260726_flash_lite`. Their SHA-256 hashes, aggregates, and figure-generation code are captured in `reproducibility/evidence_snapshot.json`.

The primary dataset is canonical HumanEval [4]. The Pro pilot uses HumanEval/0–4, and the Flash-Lite replication uses HumanEval/0–19 in canonical order. Historical context is generated deterministically (seed 42) from prompts and canonical solutions of *other* HumanEval tasks; the target task solution is excluded. Whole blocks are appended, so actual lengths exceed targets. For $n = 20$ and targets $\{256, 1024, 4096, 16384\}$, mean actual lengths are $\{371.2, 1154.8, 4210.6, 16524.3\}$ tokens, with ranges $\{259\text{--}545, 1024\text{--}1411, 4100\text{--}4415, 16389\text{--}16946\}$. This context is reproducible but synthetic.

B2 Classical-MAS invokes planner, worker, and critic separately and repeats shared context; B3 Hybrid-INoT runs the internal role cycle over one compressed-context transmission. Models are `google/gemini-3.1-pro-preview` and `google/gemini-3.1-flash-lite`. Recorded prices are \$2/\$12 and \$0.25/\$1.50 per million input/output tokens, respectively [15, 16], fixed on 26 July 2026. These API aliases are not immutable weight snapshots, so provider drift remains possible.

For each context length, B2 and B3 are paired on task and seed. We report

$$S_T = 100 \left(1 - \frac{\bar{T}_{B3}}{\bar{T}_{B2}} \right), \quad S_C = 100 \left(1 - \frac{\bar{C}_{B3}}{\bar{C}_{B2}} \right), \quad (8)$$

$$\Delta U_{\text{tok}} = 100 \left(\frac{U_{\text{tok}}^{B3}}{U_{\text{tok}}^{B2}} - 1 \right). \quad (9)$$

Continuous paired differences use two-sided Wilcoxon signed-rank tests; binary outcomes use exact McNemar tests. Percentile bootstrap confidence intervals use 10^4 resamples, and four context-length comparisons are adjusted by Holm–Bonferroni.

7.2 E3 — Scaling with context length

Table 3 reports observed API means. B2 grows with repeated context, whereas B3 remains around 5.6–6.7k tokens after the 4,096-token target because of compression.

For Pro at 16,384 target tokens,

$$S_T = 100 \left(1 - \frac{6608.4}{59631.4} \right) = 88.9\%, \quad S_C = 100 \left(1 - \frac{0.0294908}{0.1407228} \right) = 79.0\%.$$

Since $\text{pass}@1$ is one, $U_{B2} = 1/59631.4 = 1.677 \cdot 10^{-5}$ and $U_{B3} = 1/6608.4 = 1.513 \cdot 10^{-4}$, an aggregate ratio-of-means gain of 802.4%. Computing the ratio within each task before aggregation gives 805.7% (95% bootstrap CI 753.4–850.1%).

Table 3. Real API measurements for B2 and B3 on HumanEval

Model	Context	T_{B2}	T_{B3}	$S_T, \%$	$S_C, \%$	pass@1 B2/B3
Pro ($n = 5$)	256	4964	2203	55.6	43.1	1.00/1.00
	1024	7117	2925	58.9	46.8	1.00/1.00
	4096	18218	6706	63.2	49.1	1.00/1.00
	16384	59631	6608	88.9	79.0	1.00/1.00
Flash-Lite ($n = 20$)	256	2738	1379	49.6	26.0	1.00/1.00
	1024	5487	2271	58.6	41.8	1.00/1.00
	4096	16007	5664	64.6	56.9	1.00/1.00
	16384	58110	5633	90.3	86.9	1.00/1.00

B2 and B3 token use on HumanEval as context grows

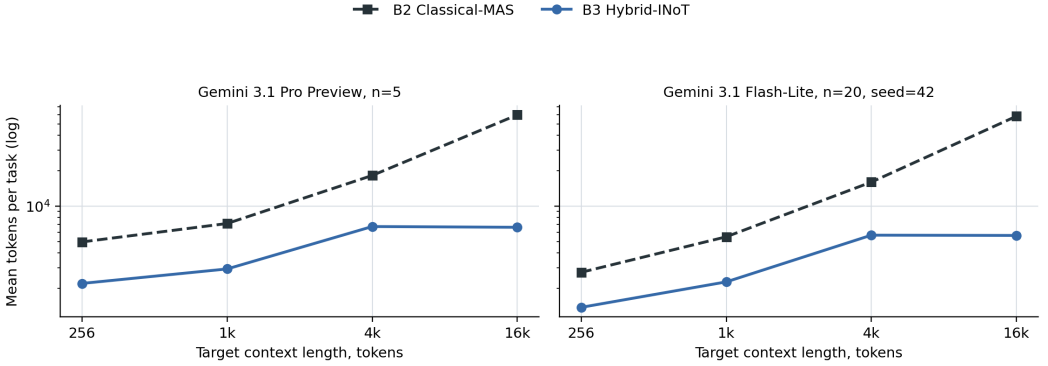


Fig. 1. Mean total tokens per task. Dashed squares denote B2 Classical-MAS; solid circles denote B3 Hybrid-INoT. The y axis is logarithmic. Left: retained Pro pilot ($n = 5$ per point); right: Flash-Lite replication ($n = 20$, seed 42).

All Pro paired gains are positive, but with five nonzero one-directional pairs the minimum two-sided p is 0.0625; none is significant after correction. In the Flash-Lite run, mean paired gains are 105.5, 144.3, 183.2, and 934.6%; all CIs exclude zero, and $p = 1.91 \cdot 10^{-6}$ at every context length, surviving Holm correction.

Nevertheless, pass@1 = 1.0 for both architectures creates a ceiling. There are no discordant pairs, exact McNemar $p = 1$, and the sample does not establish non-inferiority within 2 percentage points. Only the token component of H1 is supported. Because B3 dominates at the leftmost grid point, tau_star=256 means left censoring, $\tau^* \leq 256$ target tokens, not a point estimate.

7.3 E2 – Ablations, critic behavior, and parallelism

The Pro ablation pilot contains five tasks. pass@1 is 1.0 for full B3 and variants without compression, critic, or self-check. Paired aggregate-success differences for full B3 against the three variants are -0.0037 , -0.0262 , and 0.0000, with $p = 1.0$, 0.5, and 1.0; none survives Holm correction. Component token-overhead shares are 36.6% for compression, 29.7% for the critic, and 33.8% for self-check, but these are overhead allocations, not demonstrated independent quality contributions. Only two B0

failures were available for E2b; zero erroneous critic approvals in two cases do not establish critic reliability.

In E2c, mean B2 latency is 19.28 s and B3 latency 9.76 s, a -49.4% change; the 95% bootstrap CI for B3–B2 is $[-13.05, -6.83]$ s, but the Wilcoxon test gives $p = 0.0625$ at $n = 5$. A separate deterministic tool microbenchmark shows serial execution to be 148.8% slower than parallel execution, yet the direct architectural comparison does not show the predicted B3 penalty. H3 is therefore *not supported*; the microbenchmark only confirms a mechanism requiring isolation on real independent-tool tasks. E2d produces seven ranking inversions under weight and λ changes, with the no-critic variant ranked first canonically. The composite metric is therefore unstable and retained only as a secondary measure.

7.4 E4 – Economic test of the small model

Twenty HumanEval cases and twenty approximate SWE-bench Lite cases were run with Gemini 3.1 Pro Preview as the large model and Flash-Lite as the small model. Net margin per task is

$$M = (\rho_0 - \rho_1)C_{\text{rerun}}^L - C_{\text{check}}^S. \quad (10)$$

Table 4. Small-model profitability test, USD per task

Dataset	ρ_0	ρ_1	C_{check}^S	C_{rerun}^L	M (95% CI)	H2
HumanEval	.35	.00	.000060	.010822	.003728 [.002042; .007685]	yes
SWE-bench Lite*	.75	.60	.000122	.014469	.002048 [−.000104; .004650]	no

For HumanEval, $M = (0.35 - 0) \cdot 0.0108224 - 0.000060025 = 0.003727815$, and the lower CI bound is positive. The SWE-bench Lite interval crosses zero, so the original H2 requirement for both datasets is not met. The asterisk is essential: the current loader does not run the official containerized SWE-bench harness [11]; it uses code extraction and a `check(_)` stub. This result characterizes the harness routing/check procedure, not the ability to resolve real GitHub issues.

7.5 E6 – Cross-model sensitivity

The retained E6 compares Pro and Flash-Lite on ten controlled cases per architecture–context cell. Three simple function families are repeated (`reverse_words`, `count_peaks`, and `normalize_path`); `pass@1` is 1.0 throughout. At contexts 512, 2,048, and 8,192, Flash-Lite cost is 4.8, 6.9, and 9.7% of Pro cost for B2, and 4.4, 5.4, and 7.0% for B3 (Fig. 2). Paired U_{tok} differences are significant ($p = 0.001953$, all survive Holm correction), but the quality ceiling prevents generalization to difficult tasks.

7.6 E5 – API-inference cost extrapolation

The full TCO experiment was not completed: an additional run hit the provider limit before runs. `json` was persisted and is excluded. Instead, we perform an explicitly limited analytical extrapolation from twenty paired Pro E3 cases: five tasks at each of four context lengths with equal weights. For N tasks,

$$\Delta C_{\text{inf}}(N) = N \cdot \frac{1}{20} \sum_{i=1}^{20} (C_{B2,i} - C_{B3,i}). \quad (11)$$

The mean difference is \$0.042996 per task; at $N = 10,000$, B3 savings are \$429.96, with a 95% paired-bootstrap CI of \$267.06–\$617.17. Scaling both token prices by $-50, -25, 0, +25, +50\%$ gives \$214.98, \$322.47, \$429.96, \$537.45, and \$644.94, respectively (Fig. 3).

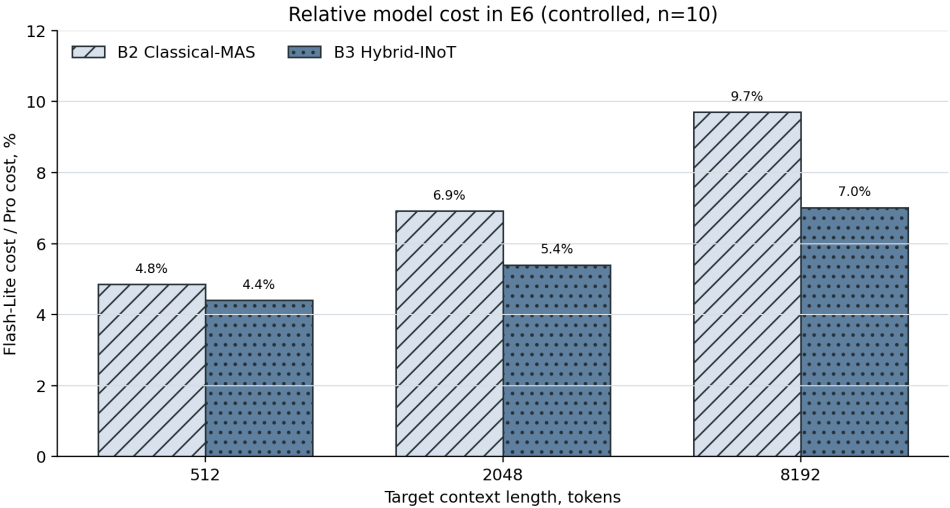


Fig. 2. Mean Gemini 3.1 Flash-Lite cost relative to Gemini 3.1 Pro Preview in controlled E6. Hatching preserves black-and-white readability.

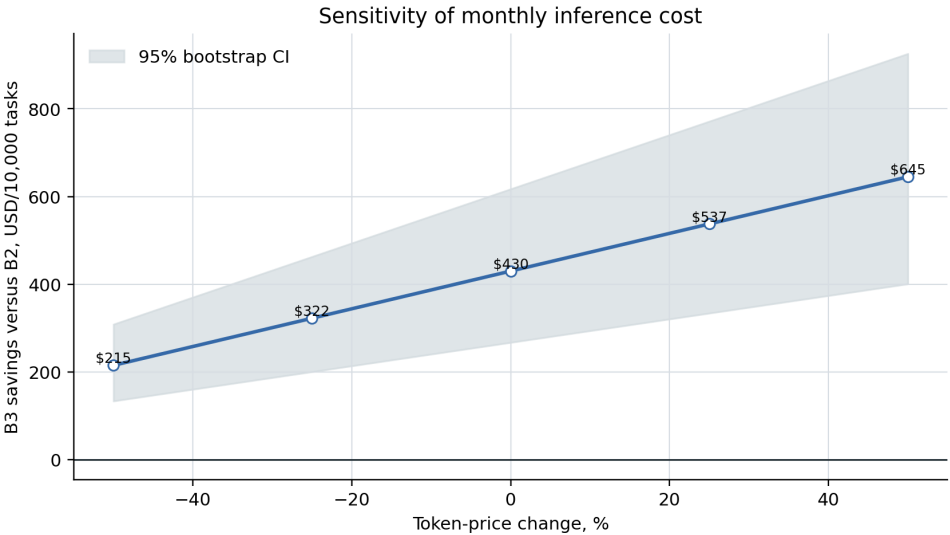


Fig. 3. Sensitivity of monthly B3 savings versus B2 to a common token-price multiplier: 10,000 tasks and an equal mixture of four context lengths. The band is the 95% paired-bootstrap CI.

8 Economic model

8.1 Boundary between inference cost and TCO

$$TCO_{\text{month}} = C_{\text{inf}} + C_{\text{hw}} + C_{\text{energy}} + C_{\text{storage}} + C_{\text{ops}}. \tag{12}$$

Only external-API C_{inf} is measured. Hardware, energy, storage, operations, engineering labor, and delay costs are not observed. Thus, E5 supports API-inference savings for the pilot mixture but not $\Delta\text{TCO} < 0$.

8.2 Hypothesis verdicts

Table 5. Original hypotheses versus observed evidence

Claim	Verdict	Evidence
H1	partial	B3 token gain is supported in Flash-Lite; Pro is underpowered, and 2-pp pass@1 non-inferiority is not established.
H2	not supported as stated	Positive CI on HumanEval, but the approximate SWE-bench Lite CI crosses zero.
H3	not supported	The tool microbenchmark favors parallelism, but direct B3 is faster than B2; $n = 5$.
τ^*	not identified	B3 dominates at the left boundary; only $\tau^* \leq 256$ target tokens is established.

9 Limitations and threats to validity

- (1) **Small, non-random samples.** The first 5 or 20 HumanEval tasks are used rather than a random stratified sample of 164; the new run has one seed.
- (2) **Synthetic context.** Long inputs are assembled from other HumanEval tasks rather than repository history. They isolate retransmission cost but not real codebase dependencies.
- (3) **Quality ceiling.** B2/B3 and controlled E6 have pass@1 = 1.0, supporting a resource comparison but not a strong quality test.
- (4) **Approximate SWE-bench.** The official container harness is not used, so no claim about real issue resolution is made.
- (5) **API drift.** Model names, parameters, usage, latency, and cost are recorded, but aliases do not guarantee immutable weights. Provider-limit failures are excluded.
- (6) **Limited answer auditability.** runs.json records metrics and usage but not complete final solution text.
- (7) **Cost is not TCO.** Figure 3 covers API inference only; the other terms in (12) are unmeasured.
- (8) **Architectural boundaries.** Independent subtasks, process isolation, critic errors, and compression losses may eliminate the B3 advantage. Role text is a computational mechanism, not a faithful explanation of hidden reasoning [25].

10 Directions for further research

The next study should remove the identified uncertainty rather than repeat the pilot:

- (1) run all 164 HumanEval tasks with at least five seeds and a fixed analysis plan, including a powered non-inferiority test;
- (2) extend the context grid below 256 tokens to estimate rather than left-censor τ^* ;
- (3) retain full input/output, prompt hashes, immutable model identifiers, and external-test results per run;
- (4) replace the approximate SWE-bench path with the official container harness on a random repository sample;
- (5) repeat E2c with real independent tools under matched models and serial/parallel execution;

- (6) compare B3 with Self-Refine [13], Reflexion [22], ReAct [30], and SWE-agent [29];
- (7) measure full TCO, including operations, reruns, latency, infrastructure, and storage, under a production context-length distribution.

11 Conclusion

Hybrid-INoT is formalized as an intermediate architecture between in-context self-critique and external multi-agent orchestration. The analytical model predicts a growing advantage with shared-context length, and two real API pilots agree with that prediction. In the 20-task Flash-Lite replication, B3 uses 49.6–90.3% fewer tokens than B2, and paired U_{tok} gains remain significant after multiple-comparison correction. However, ceiling pass@1 does not establish quality non-inferiority; H2 is supported only on HumanEval; H3 is not supported in the direct pilot; and the \$429.96 per 10,000 tasks estimate covers API inference only. The evidence therefore supports Hybrid-INoT as a promising mechanism for reducing repeated long-context transmission, while quality, the applicability threshold, and full economic impact require a powered evaluation on real repositories.

Data Availability

An anonymous replication package, `hybrid-inot-july-2026-anonymous.zip`, accompanies this manuscript. It contains the frozen research harness, configurations, recorded metrics and usage, and scripts and source data for the reported tables and figures. The records do not include complete generated solution text; this limits answer-level auditing. The package supports inspection and offline analysis of retained results; rerunning hosted models may require an API account and may be affected by model drift. The replication materials will be made publicly available upon acceptance.

References

- [1] Anonymous authors. 2026. Hybrid-INoT replication package. Anonymous review artifact supplied with this manuscript.
- [2] J. Austin et al. 2021. Program Synthesis with Large Language Models. arXiv:2108.07732.
- [3] Z. Bi et al. 2024. Iterative Refinement of Project-Level Code Context for Precise Code Generation with Compiler Feedback. arXiv:2403.16792.
- [4] M. Chen et al. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374.
- [5] M. K. Goncalves, C. R. B. de Souza, and V. M. Gonzalez. 2011. Collaboration, Information Seeking and Communication. *J. Universal Computer Science* 17, 11 (2011), 1529–1550.
- [6] T. Guo et al. 2024. Large Language Model based Multi-Agents: A Survey of Progress and Challenges. arXiv:2402.01680.
- [7] S. Hong et al. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *ICLR*.
- [8] ISO/IEC. 2023. *Systems and software engineering — SQuARE — Product quality model*. Standard ISO/IEC 25010:2023. ISO.
- [9] H. Jia, E. T. Barr, and S. Mechtaev. 2026. Compressing Code Context for LLM-based Issue Resolution. arXiv:2603.28119.
- [10] H. Jiang et al. 2024. LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression. In *ACL*.
- [11] C. E. Jimenez et al. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *ICLR*.
- [12] A. J. Ko, R. DeLine, and G. Venolia. 2007. Information Needs in Collocated Software Development Teams. In *Proc. 29th Int. Conf. Software Engineering*. 344–353.
- [13] A. Madaan et al. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *NeurIPS*.
- [14] T. J. McCabe. 1976. A Complexity Measure. *IEEE Trans. Software Engineering* SE-2, 4 (1976), 308–320.
- [15] OpenRouter. 2026. Gemini 3.1 Flash Lite Preview — pricing. <https://openrouter.ai/google/gemini-3.1-flash-lite/pricing> Accessed: 26 July 2026.
- [16] OpenRouter. 2026. Gemini 3.1 Pro Preview — pricing. <https://openrouter.ai/google/gemini-3.1-pro-preview/pricing> Accessed: 26 July 2026.
- [17] Z. Pan et al. 2024. LLMingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression. In *ACL Findings*.
- [18] C. Parnin and S. Rugaber. 2011. Resumption Strategies for Interrupted Programming Tasks. *Software Quality Journal* 19, 1 (2011), 5–34.

[19] C. Qian et al. 2024. ChatDev: Communicative Agents for Software Development. In *ACL*.
[20] C. Qian et al. 2024. Scaling Large Language Model-based Multi-Agent Collaboration. *arXiv:2406.07155*.
[21] M. Sepidband, H. Taherkhani, S. Wang, and H. Hemmati. 2025. Enhancing LLM-Based Code Generation with Complexity Metrics. *arXiv:2505.23953*.
[22] N. Shinn et al. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *NeurIPS*.
[23] C.-Y. Su and C. McMillan. 2025. Do Code LLMs Do Static Analysis? *arXiv:2505.12118*.
[24] H. Sun and S. Zeng. 2025. Introspection of Thought Helps AI Agents. *arXiv preprint. arXiv:2507.08664*.
[25] M. Turpin, J. Michael, E. Perez, and S. R. Bowman. 2023. Language Models Don't Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting. In *NeurIPS*.
[26] J. Wei et al. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *NeurIPS*, Vol. 35. 24824–24837.
[27] G. Weiss. 1999. *Multiagent Systems*. MIT Press.
[28] M. Wooldridge. 2009. *An Introduction to MultiAgent Systems* (2 ed.). Wiley.
[29] J. Yang et al. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *NeurIPS*.
[30] S. Yao et al. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *ICLR*.